CampusNetSim: A Cloud-Native Wi-Fi Congestion Simulation and Automated Load-Balancing System on AWS Serverless Services

Sumedha Phadnis and Priyanka J

Cloud Architecture Design (AWS) - Course Project Report

Keywords—Serverless computing, AWS Lambda, Amazon API Gateway, Amazon DynamoDB, DynamoDB Streams, event-driven architecture, WLAN load balancing, access point utilization, campus network simulation.

***Abstract*—Campus Wi-Fi load is rarely even: client devices associate with the strongest access point (AP), so APs near lecture halls saturate while neighbouring APs stay lightly used. This paper presents CampusNetSim, a cloud-native system that simulates AP association on a campus, detects overloaded APs and automatically rebalances simulated clients onto APs with spare capacity. The backend is fully serverless: ten AWS Lambda functions behind an Amazon API Gateway HTTP API, four on-demand Amazon DynamoDB tables, and a DynamoDB Stream that drives load balancing and history logging without any polling. Real phones join the simulation by scanning an AP's QR code, which opens a check-in page hosted on Amazon S3, and a React dashboard on AWS Amplify shows live utilization, migrations, connected devices and traffic trends. The final build adds run-time capacity reconfiguration through a validated PUT endpoint and makes event processing concurrency-safe with conditional writes, so that DynamoDB becomes the single source of truth for the campus layout. In two live runs with real phones, the load balancer reacted to an AP at 100% utilization and moved a device to the AP with the most free capacity, bringing the source below the 85% trigger and leaving the target at 33.33%. All 14 functional API tests returned the expected status codes, and warm read requests completed with a median latency of 60-69 ms.**

I. Introduction

A crowded lecture hall at a class change shows a familiar weakness of campus Wi-Fi. Client radios associate with the access point that offers the strongest signal, regardless of how many clients that AP already serves. One AP saturates while an AP in the library, two buildings away, sits half empty. Research on WLAN association control has shown that signal-strength-only association produces markedly worse throughput and fairness than load-aware association [1]-[3].

Moving clients to a less loaded AP is harder than it looks. The IEEE 802.11 standard [4] lets an AP suggest a transition through BSS Transition Management [5] and speeds up the roam once the client decides to move [6], but the decision always remains with the client's chipset and driver. Consumer routers rarely expose steering controls to third-party software, and mobile operating systems do not let applications switch networks silently. A student project therefore cannot enforce real client steering reliably across heterogeneous devices.

CampusNetSim addresses the problem inside a controlled simulation. Four APs (AP01-AP04) are modelled across three buildings: two in the academic block, one in the hostel and one in the library. Each AP has a configurable capacity, and every association change is an explicit CONNECT or DISCONNECT event. Because the system owns the association state, it can detect an overloaded AP, choose a target, perform the move and record verifiable before-and-after utilization. Real phones still take part: scanning an AP-specific QR code opens a browser check-in page that registers the phone with that AP in application state, while the phone's actual network connection is never changed.

The system is built entirely on managed AWS serverless services [7]-[9]. The main contributions are: (i) an event-driven, polling-free load-balancing pipeline built on DynamoDB Streams; (ii) run-time capacity reconfiguration with server-side validation and concurrency-safe event processing; (iii) a phone check-in path that lets real devices trigger the pipeline without any installation; and (iv) a live evaluation with real phones, functional API tests and latency measurements.

The rest of the paper is organised as follows. Section II reviews related work. Section III presents the methodology, Sections IV and V describe the architecture and workflows, and Section VI formalises the utilization model and the balancing algorithm. Section VII summarises the system configuration in tables, Section VIII reports the implementation and results, and Sections IX and X conclude and outline future work.

II. Related Works

Load balancing in wireless LANs has been studied mainly at the level of AP association. Bejerano, Han and Li [1] formulated association control for max-min fair bandwidth allocation and showed that load-aware association clearly outperforms the default strongest-signal policy. Bejerano and Han [2] later proposed cell breathing, in which APs adjust their transmit power to shrink or grow their coverage so that clients drift toward less loaded cells without any client-side change. Balachandran, Bahl and Voelker [3] studied hot-spot congestion in public-area wireless networks and proposed network-directed roaming, in which an overloaded AP explicitly redirects users to neighbouring APs. CampusNetSim follows the same idea of network-side redirection, but performs it on a simulated association model where every move can be enforced and audited.

The 802.11 standard [4] defines how roaming can be influenced in real deployments. The 802.11v amendment [5] introduced BSS Transition Management, through which an AP can request that a client move, and 802.11r [6] defines Fast BSS Transition to reduce roaming latency. Both remain advisory from the network's side: a client may ignore a transition request. This is the main reason CampusNetSim enforces moves inside its own event model rather than attempting real client steering.

On the cloud side, Jonas et al. [7] and Castro et al. [8] describe serverless computing as a model in which developers deploy stateless functions while the provider handles provisioning and scaling, at the cost of cold starts and platform-specific integration. The survey by Li et al. [9] classifies serverless architectures and design trade-offs in detail. Amazon's Dynamo paper [10] motivates key-value stores such as DynamoDB for workloads dominated by simple key-based reads and writes, which matches per-AP state, append-only history and migration logs. Ahmed et al. [11] built a four-layer IoT platform for remote monitoring and control of smart campus buildings; CampusNetSim shares the goal of live campus monitoring through a web dashboard but drives it from simulated association events and phone check-ins instead of physical sensors. Table I compares the approaches.

TABLE I
COMPARISON OF RELATED APPROACHES

Approach	Where control lives	Needs client support	Relation to CampusNetSim
Association control [1]	Network (central)	Yes	Same objective; simulated enforcement
Cell breathing [2]	AP transmit power	No	Alternative real-network mechanism
Network-directed roaming [3]	Overloaded AP	Partly	Closest idea: redirect from hot spot
802.11v / 802.11r [5], [6]	AP request, client decides	Yes	Advisory only; motivates simulation
Smart-campus IoT [11]	Cloud platform	Sensors	Similar cloud monitoring stack

III. PROPOSED METHODOLOGY

A. System Model

The campus is modelled as a set of APs, each with an identifier, a building, a display name, a capacity C_i (the number of devices at which the AP is considered full) and a live connected-device count n_i. Association changes arrive as CONNECT and DISCONNECT events from three sources: phones using the check-in page, the load balancer when it moves devices, and an optional synthetic traffic generator. The state of every AP is held in one DynamoDB item, so every change to an AP is a single item update that DynamoDB Streams can publish [14].

B. Design Principles

Four principles guided the design. **One function per responsibility:** each API route has its own Lambda function with permissions limited to the tables it needs, following the AWS serverless guidance [12]. **Event-driven reaction:** load balancing and history logging are triggered by the APState stream rather than by a schedule, so the system is idle when nothing changes [13], [14]. **Single source of truth:** AP capacities, buildings and names live only in the APState table; every function reads them from there at run time. **Verifiable actions:** every automated move writes a Migrations record with the utilization of both APs before and after the move.

C. Functional Components

Event ingestion. EventProcessor handles POST /events. It validates the event, reads the AP's configuration and current count with a strongly consistent read, rejects a CONNECT when the AP is already full, computes the new utilization and status, and writes only the live-state attributes back with a conditional update. When an AP becomes CONGESTED, it also attaches an advisory recommendation that prefers an AP in the same building.

Reactive load balancing. LoadBalancer consumes the APState stream. When an AP reaches the 85% trigger, it selects the AP with the most free capacity, computes how many devices to move (at most five) and moves them by posting DISCONNECT and CONNECT events through the public API, so every move passes the same validation as any other event. Real phones are moved first, and their roster entries are updated so that the phone's page can show the redirect. HistoryWriter consumes the same stream and stores each AP state in APHistory with a 24-hour expiry using DynamoDB time to live (TTL) [15].

Read services and administration. StatusReader, HistoryReader and MigrationReader serve the dashboard and the check-in page. DeviceRegistry stores the phone roster. AdminReset zeroes all APs, clears the Migrations and APHistory tables and marks every connected phone as disconnected, so a demonstration can be repeated from a clean state.

D. Run-Time Capacity Reconfiguration and Consistency

The final build lets an operator change an AP's capacity while the system is running. A new UpdateAPConfig function behind PUT /aps/{ap_id}/config accepts only the fields capacity, building and ap_name, requires the capacity to be an integer from 1 to 1000, and refuses a capacity lower than the number of devices currently connected (HTTP 409). It updates only the supplied attributes and recalculates the AP's utilization in the same write. Because EventProcessor now reads capacity from APState on every event, the new value applies to the very next event, including the utilization status and the balancing trigger.

Two concurrent events on the same AP could otherwise read the same count and overwrite each other. EventProcessor therefore uses optimistic concurrency [16]:

its update succeeds only if the stored count and capacity still equal the values it read; otherwise DynamoDB rejects the write and the API returns HTTP 409 so that the caller can retry. The same condition protects against a capacity change that lands between the read and the write of an event.

IV. Architecture Diagram

Fig. 1 shows the deployed architecture in five layers. The client layer contains the phone check-in page, a static page on Amazon S3, and the monitoring dashboard, a React and Vite application on AWS Amplify Hosting. Both call a single API Gateway HTTP API [17] with CORS enabled. The compute layer has seven request handlers, one per API route, two event-driven functions attached to the APState stream, and TrafficGenerator, which is deployed without a trigger and is invoked manually for synthetic load. The storage layer consists of four on-demand DynamoDB tables, and every function logs to Amazon CloudWatch Logs.

Only two kinds of connection carry the system's logic: request handlers read and write tables in response to API calls, and the APState stream fans each change out to LoadBalancer and HistoryWriter. LoadBalancer writes moves back through the API instead of writing APState directly, which keeps all capacity and concurrency checks in EventProcessor.

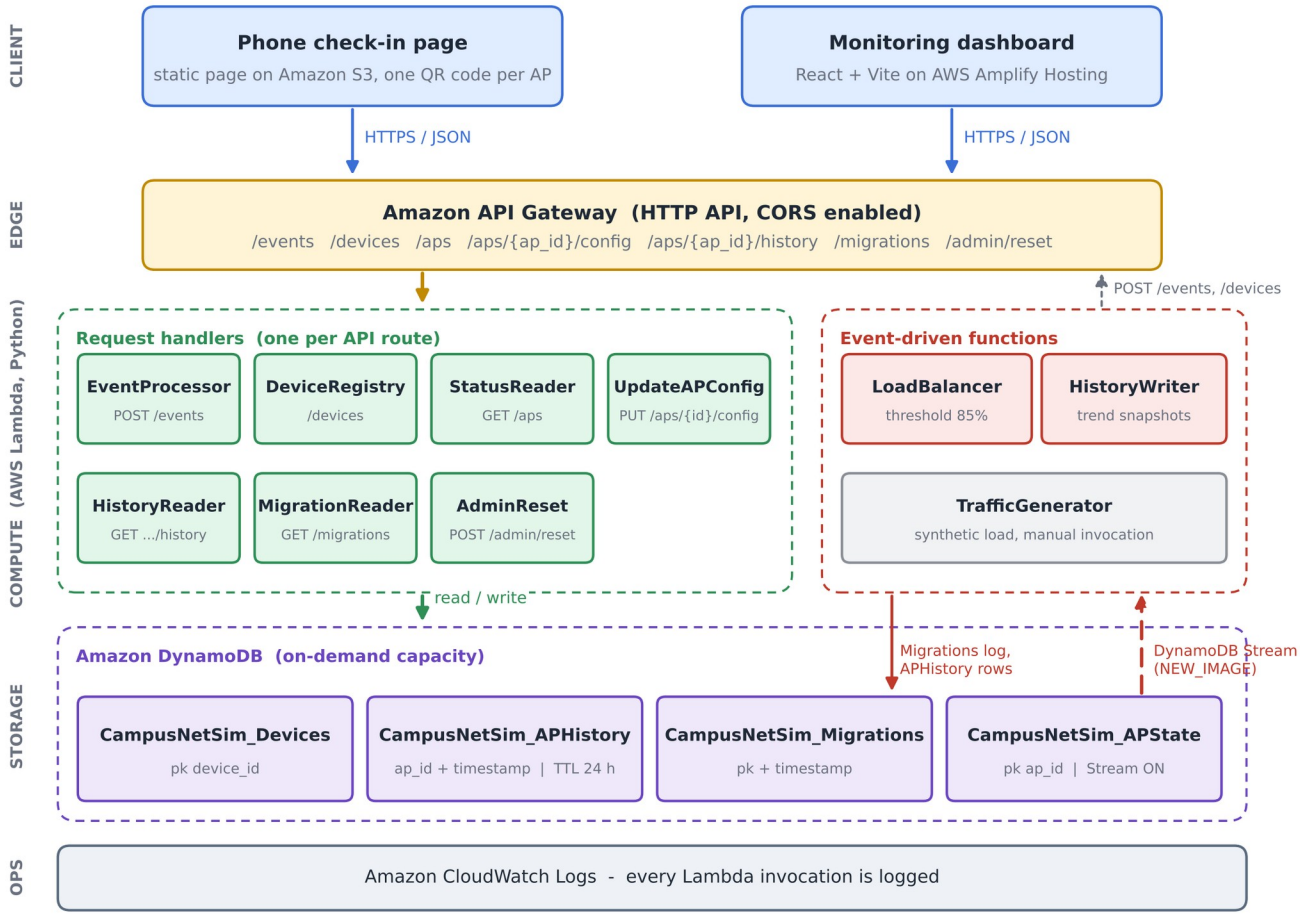


Fig. 1. CampusNetSim system architecture. Request handlers serve the seven API routes, the APState stream drives the two event-driven functions, and all state lives in four on-demand DynamoDB tables.

V. Workflow

Fig. 2 traces one device from check-in to the refreshed dashboard. Scanning the QR code opens the check-in page with the AP identifier in the link (step 1). Tapping Join posts a CONNECT event to /events and then a JOIN to /devices (step 2). EventProcessor validates the event and updates APState (step 3), which emits one stream record (step 4). The record reaches LoadBalancer and HistoryWriter independently (steps 5 and 6). LoadBalancer acts only when the AP has reached 85%, moving devices and logging the migration (step 7), while HistoryWriter always stores a snapshot for the trend chart. Both front ends poll the read endpoints every five seconds (step 8); a migrated phone notices its new AP in the roster and shows a banner explaining the move.

Fig. 3 shows the capacity reconfiguration flow. The operator selects an AP on the dashboard and enters a new maximum, which the dashboard checks locally before

sending PUT /aps/{ap_id}/config. UpdateAPConfig validates the request again on the server and updates APState in place. Because that update also passes through the APState stream, the trend chart records the recalculated utilization immediately.

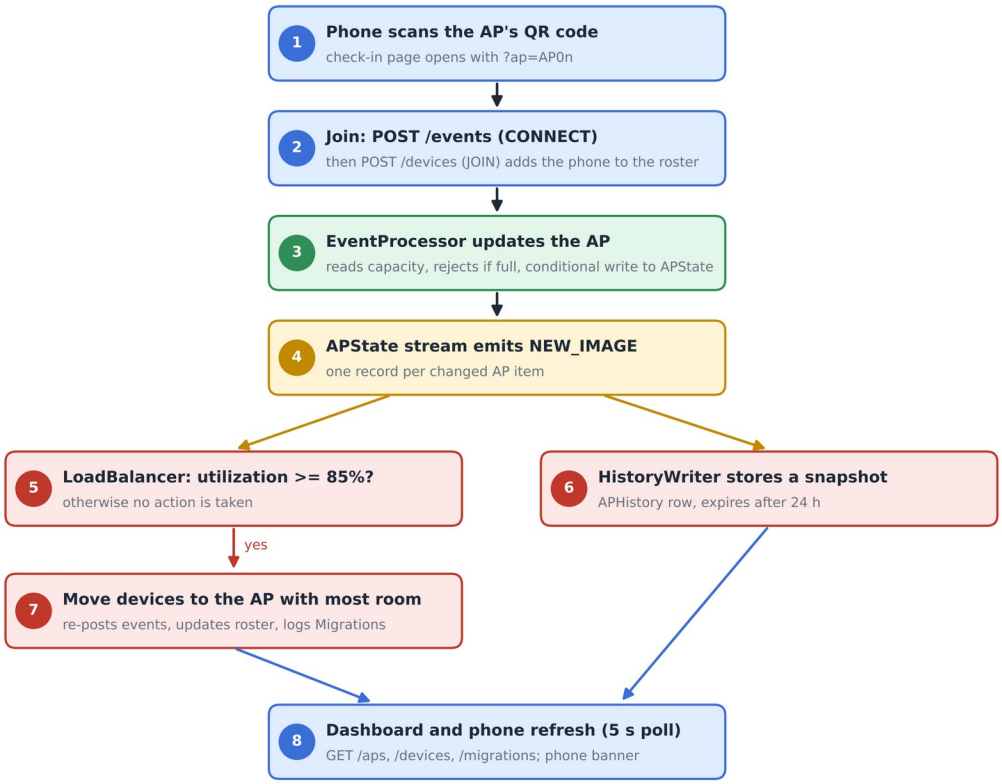


Fig. 2. End-to-end workflow from phone check-in to automated rebalancing and dashboard refresh.

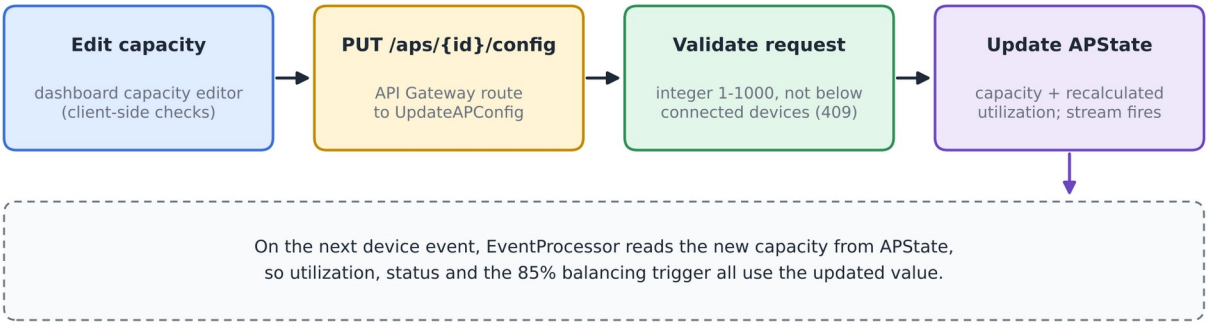


Fig. 3. Run-time capacity reconfiguration flow.

VI. Algorithm / Formulas

A. Utilization and Status

For an AP i with n_i connected devices and capacity C_i, utilization is

$$U_i = (n_i / C_i) \times 100 \quad (1)$$

rounded to two decimals. The status shown on the dashboard and the check-in page is

$$S_i = NORMAL\ (U_i < 70),\ BUSY\ (70 \le U_i < 90) \quad (2)$$

with HIGH_LOAD for $90 \le U_i < 100$ and CONGESTED for $U_i \ge 100$. Because a CONNECT is rejected once $n_i = C_i$, utilization never exceeds 100%, and CONGESTED means exactly full.

B. Balancing Trigger, Target and Move Size

LoadBalancer reacts to a stream record for AP i only when

$$U_i \ge \theta, \quad \theta = 85 \quad (3)$$

and the condition still holds in a fresh read of GET /aps. The target is the AP with the most free slots,

$$j^* = argmax_{j \neq i} (C_j - n_j) \quad (4)$$

and the source should drop to about five points below the trigger, giving a target count

$$n_i^{tgt} = \lfloor C_i \, (\theta - 5) / 100 \rfloor \quad (5)$$

The number of devices moved is limited by the need, the free space at j* and a per-migration cap K = 5:

$$k = min(\, max(1, \, n_i - n_i^{tgt}), \, C_{j^*} - n_{j^*}, \, K \,) \quad (6)$$

C. Capacity Update and Consistency Rules

A capacity change for AP i to a new value C_i' is accepted only if

$$n_i \leq C_i' \leq 1000, \quad C_i' \in \mathbb{Z} \quad (7)$$

and the stored utilization is recomputed with (1). An event write computed from a read of (n_i, C_i) is committed only if the stored item still matches that read:

$$commit \Leftrightarrow n_i^{stored} = n_i^{read} \wedge C_i^{stored} = C_i^{read} \quad (8)$$

As a worked example, consider an AP with $C_i = 2$ and $n_i = 2$ ($U_i = 100\%$). Equation (5) gives $n_i^{tgt} = \lfloor 2 \times 80 / 100 \rfloor = 1$, so (6) moves k = 1 device, provided the target has at least one free slot. The source then reaches 50%. This matches Run A in Section VIII.

D. Algorithm

Algorithm 1 lists the procedure that LoadBalancer runs for each stream record, and Fig. 4 shows the same logic as a flowchart.

Algorithm 1: Reactive AP load balancing (per stream record)

1: *input:* stream record r for AP i
2: **if** r is not INSERT/MODIFY **or** device id starts with "migrated-" **then return**
3: **if** $U_i < \theta$ in r **then return**
4: read all APs via GET /aps; **if** $U_i < \theta$ **then return**
5: $j^* \leftarrow \text{argmax}_{j \neq i} (C_j - n_j)$ (4); **if** $C_{j^*} - n_{j^*} \leq 0$ **then return**
6: $k \leftarrow \min(\max(1, n_i - n_i^{tgt}), C_{j^*} - n_{j^*}, 5)$ (5)-(6)
7: D ← connected real phones on AP i (roster)
8: **for** m = 1 **to** k **do**
9: d ← D[m] if available, else synthetic id "migrated-i-xxxx"
10: POST /events (d, i, DISCONNECT); POST /events (d, j*, CONNECT)
11: **if** d is a real phone **then** POST /devices (d, JOIN, j*)
12: **if** at least one move succeeded **then** write Migrations record {i, j*, moved, U before/after}

The guard in line 2 prevents feedback loops: events produced by a migration change APState again, but records written for synthetic migrated devices are ignored, and a target AP that receives devices is normally below the trigger. Because moves go through POST /events, the capacity check and the conditional write in EventProcessor also protect the balancer's own writes.

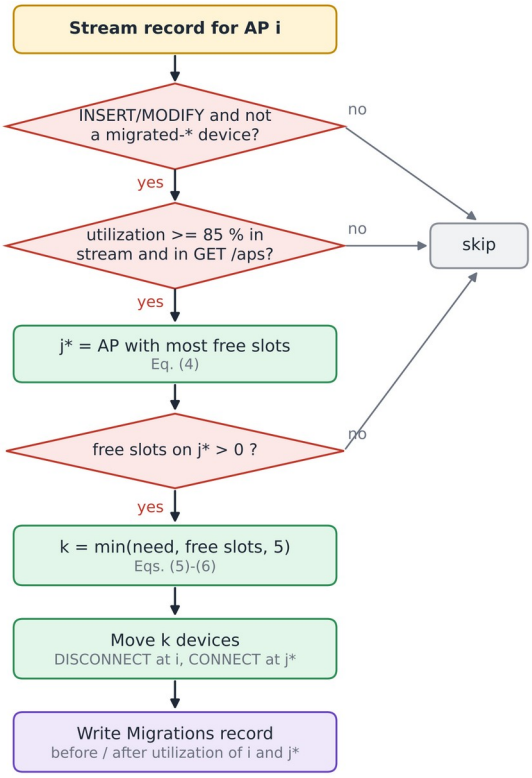


Fig. 4. Decision flowchart of the LoadBalancer function (Algorithm 1).

VII. Tables and Figures

This section collects the configuration of the deployed system. Table II lists the Lambda functions with their triggers and settings, Table III the DynamoDB tables, Table IV the API routes and Table V the parameters used in Section VI. Figures 1-4 above describe the architecture, workflows and algorithm; Figs. 5-10 in Section VIII document the running system and the measured results.

TABLE II
AWS Lambda Functions (ap-south-1, x86_64, 128 MB)

Function	Trigger	Runtime / timeout	Responsibility
CampusNetSim-EventProcessor	HTTP API: /events	Python 3.13 / 3 s	Validate CONNECT/DISCONNECT, conditional APState update, advisory recommendation
CampusNetSim-DeviceRegistry	HTTP API: /devices	Python 3.13 / 3 s	Phone roster: JOIN, LEAVE, list
CampusNetSim-StatusReader	HTTP API: /aps	Python 3.13 / 3 s	Live state of all APs
UpdateAPConfig	HTTP API: /aps/{ap_id}/config	Python 3.14 / 3 s	Validated capacity, building and name update
CampusNetSim-HistoryReader	HTTP API: /aps/{ap_id}/history	Python 3.13 / 3 s	Up to 200 history points for one AP

Function	Trigger	Runtime / timeout	Responsibility
CampusNetSim-MigrationReader	HTTP API: /migrations	Python 3.13 / 3 s	50 most recent migrations
CampusNetSim-AdminReset	HTTP API: /admin/reset	Python 3.13 / 40 s	Reset APs, clear logs, disconnect phones
CampusNetSim-LoadBalancer	DynamoDB Stream (APState)	Python 3.13 / 30 s	Detect >= 85% and move devices
CampusNetSim-HistoryWriter	DynamoDB Stream (APState)	Python 3.13 / 3 s	Append snapshot to APHistory
CampusNetSim-TrafficGenerator	None (manual)	Python 3.13 / 30 s	Synthetic CONNECT/DISCONNECT traffic

TABLE III
AMAZON DYNAMODB TABLES (ON-DEMAND CAPACITY)

Table	Key schema	Purpose / features
APState	ap_id	Live AP config and counts; stream (NEW_IMAGE)
APHistory	ap_id + timestamp	Utilization snapshots; TTL 24 h
Devices	device_id	Phone roster and current AP
Migrations	pk + timestamp	Audit log of automated moves

TABLE IV
API GATEWAY ROUTES

Route	Method	Function
/events	POST	EventProcessor
/devices	GET, POST	DeviceRegistry
/aps	GET	StatusReader
/aps/{ap_id}/config	PUT	UpdateAPConfig
/aps/{ap_id}/history	GET	HistoryReader
/migrations	GET	MigrationReader
/admin/reset	POST (else 405)	AdminReset

TABLE V
MODEL PARAMETERS

Symbol	Value	Meaning
BUSY / HIGH_LOAD / CONGESTED	70 / 90 / 100 %	Status thresholds, Eq. (2)
θ	85 %	Balancing trigger, Eq. (3)
$\theta - 5$	80 %	Source target after a move, Eq. (5)
K	5	Max devices per migration, Eq. (6)
C_i range	1-1000	Allowed capacity, Eq. (7)
History TTL	24 h	APHistory retention
Poll interval	5 s	Dashboard and check-in refresh

VIII. IMPLEMENTATION, RESULTS, AND DISCUSSION

A. Backend Deployment

The backend runs in the AWS Mumbai region (ap-south-1). The four tables were created with on-demand capacity, so no throughput had to be provisioned. The APState stream uses the NEW_IMAGE view type and has two enabled consumers, LoadBalancer and HistoryWriter, each with a batch size of 100 starting from the latest record [13], [14]. APHistory has TTL enabled on its expires_at attribute, so old readings disappear without a clean-up job [15]. All functions are written in Python and use the boto3 SDK; read functions convert DynamoDB Decimal values back to JSON numbers. The complete source of every deployed function, the check-in page, the dashboard and an AWS SAM template that recreates the stack are published in the project repository [19].

B. Front Ends and Hosting

The phone check-in page is a single HTML file on Amazon S3. On first use it generates a device identifier with the browser's random UUID generator and keeps it in local storage, so later check-ins from the same phone map to the same roster entry. After joining, the page polls GET /devices and GET /aps every five seconds; it detects a migration when its roster entry shows a new AP, and a reset when the entry becomes DISCONNECTED, and shows an explanatory banner in both cases.

The dashboard is built with React, Vite, Tailwind CSS and Recharts. It shows the AP cards, a utilization bar chart with the 70% and 90% reference lines, the automated load-balancing log, the connected-device roster and the traffic trend of the selected AP. The final build adds the capacity editor and an AP details panel that lists the devices on the selected AP (Fig. 5). The production build is deployed to AWS Amplify Hosting by manual upload [18]. Two deployment issues had to be fixed: a ZIP archive created on Windows stored backslash paths, which made the hosted page request its JavaScript and CSS files at wrong URLs, and the Vite configuration initially lacked the Tailwind CSS plugin, which left the page unstyled. A small script now rebuilds the archive with forward-slash paths.

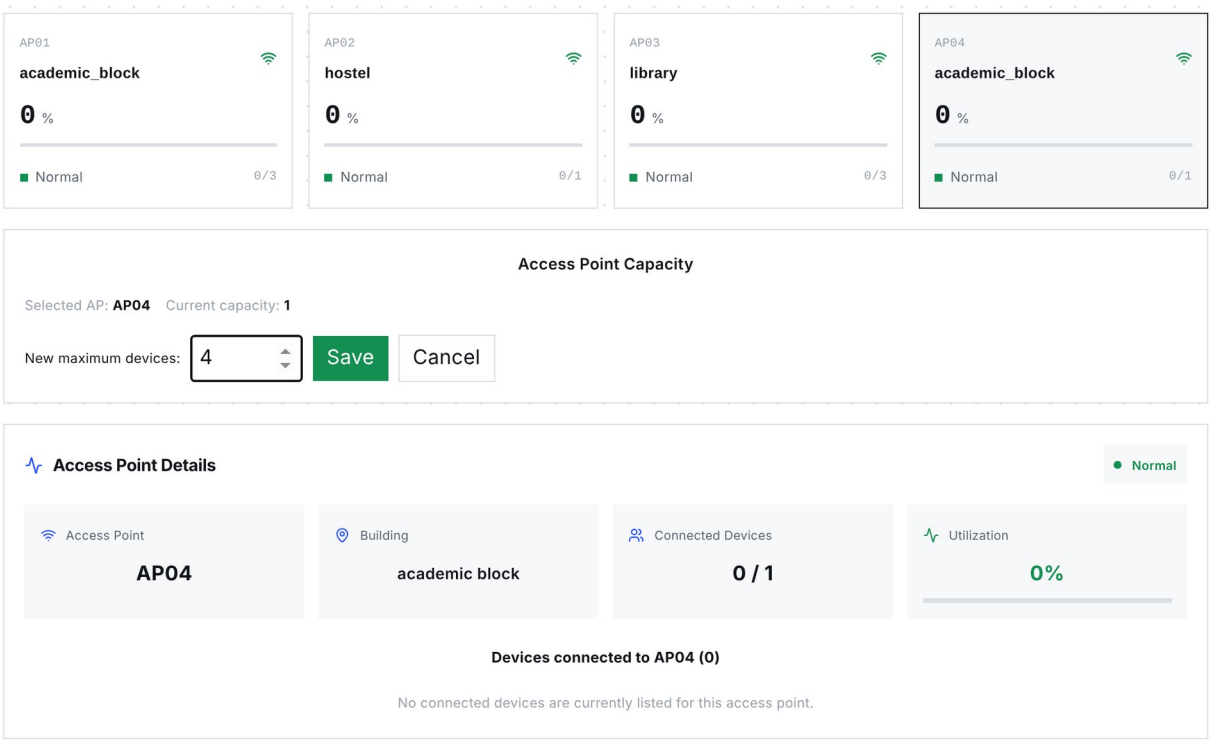


Fig. 5. Dashboard of the final build: AP cards with the configured capacities (3, 1, 3, 1), the capacity editor open for AP04, and the AP details panel for the selected AP. Rendered from the production build with the API state recorded on 23 October 2026.

C. Capacity Reconfiguration and Consistency Enhancements

Table VI summarises the changes made in the final build. Together they remove hard-coded capacities from the processing path, make concurrent writes safe and give the operator direct control over the campus layout.

TABLE VI
ENHANCEMENTS IN THE FINAL BUILD

Component	Change	Effect
UpdateAPConfig (new Lambda) and PUT /aps/{ap_id}/config	Validated partial update of capacity, building and AP name; utilization recalculated with the capacity	Operators change capacity at run time without touching DynamoDB
EventProcessor	Reads capacity, building and name from APState on every event (strongly consistent read)	DynamoDB is the single source of truth for the layout
EventProcessor	Rejects CONNECT at capacity (409); conditional update on count and capacity (409 on conflict)	No lost updates and no utilization above 100%
EventProcessor	Updates only live-state attributes instead of replacing the item	Configuration fields survive device events
Dashboard	Capacity editor with local checks (positive integer, not below connected devices) and AP details panel	Capacity changes and per-AP devices visible in one place
Dashboard deployment	Forward-slash ZIP packaging script; Tailwind plugin in the Vite configuration	Reliable Amplify deployments

D. Functional Verification

Every route was exercised against the deployed API on 23 October 2026. The tests deliberately target the validation paths, which change no data, together with the read endpoints. Table VII lists the requests and responses; all 14 returned the expected status code and error message. The reset endpoint was checked only with a GET, which is correctly refused, because a POST permanently deletes the logs.

TABLE VII
FUNCTIONAL API TESTS AGAINST THE DEPLOYED SYSTEM

#	Request	Expected	Observed response
1	GET /aps	200, four APs	200, AP01-AP04 with capacities 3, 1, 3, 1
2	GET /devices	200, roster	200, roster list
3	GET /migrations	200, log	200, one migration (AP04 to AP03)
4	GET /aps/AP03/history	200	200, empty list (readings older than 24 h expired)
5	GET /admin/reset	405	405 "Use POST to trigger a reset"
6	PUT /aps/AP01/config {capacity: 0}	400	400 "Capacity must be an integer from 1 to 1000"
7	PUT /aps/AP01/config {capacity: "four"}	400	400, same message
8	PUT /aps/AP01/config {capacity: 5000}	400	400, same message
9	PUT /aps/AP01/config {color: "red"}	400	400 "Unsupported configuration field"
10	PUT /aps/AP99/config {capacity: 4}	404	404 "AP not found"
11	POST /events, event_type PING	400	400 "event_type must be CONNECT or DISCONNECT"
12	POST /events, no device_id	400	400 "device_id, ap_id and event_type are required"
13	POST /events, ap_id AP99	400	400 "Unknown ap_id: AP99"
14	POST /devices JOIN without ap_id	400	400 "ap_id required for JOIN"

Response times of the three most frequently polled routes were measured from a browser client in India with 20 sequential requests per route after the functions were warm (Table VIII). Median latencies of 60-69 ms are far below the five-second polling interval. The first requests after a period of inactivity took up to about 1.3 s, which reflects Lambda cold starts [8].

TABLE VIII
WARM REQUEST LATENCY (20 REQUESTS PER ROUTE)

Route	Min (ms)	Median (ms)	P90 (ms)	Max (ms)
GET /aps	57	68	79	84
GET /devices	56	69	78	81
GET /migrations	58	60	80	282

E. Live Load-Balancing Runs

The pipeline was tested end to end with real phones instead of staged data. In **Run A** (September 2026), AP02 in the hostel had a capacity of 2 and AP04 in the academic block a capacity of 3. Two team members joined AP02 from their phones within a few minutes, driving it to 2 of 2 devices and 100% utilization (Fig. 8a). The resulting APState write reached LoadBalancer through the stream; it chose AP04, which had the most free slots, and moved one phone there. AP02 dropped to 50% and AP04 rose to 33.33%. The moved phone displayed the redirect banner with its new AP (Fig. 8b), and the dashboard logged the migration and showed both phones in the roster (Figs. 6 and 7).

Run B took place on 20 October 2026 during the final demonstration, after the capacities had been changed through the new capacity editor to 3, 1, 3 and 1 for AP01-AP04. Several participants checked in from their own phones. When a phone joined AP04 (capacity 1), the AP reached 100%, and LoadBalancer moved the phone to AP03 in the library, leaving AP04 at 0% and AP03 at 33.33% (Fig. 9). This run also confirms that a capacity set at run time is used by EventProcessor and by the balancing trigger without redeploying anything. Table IX lists both migrations and Fig. 10 compares utilization before and after each move.

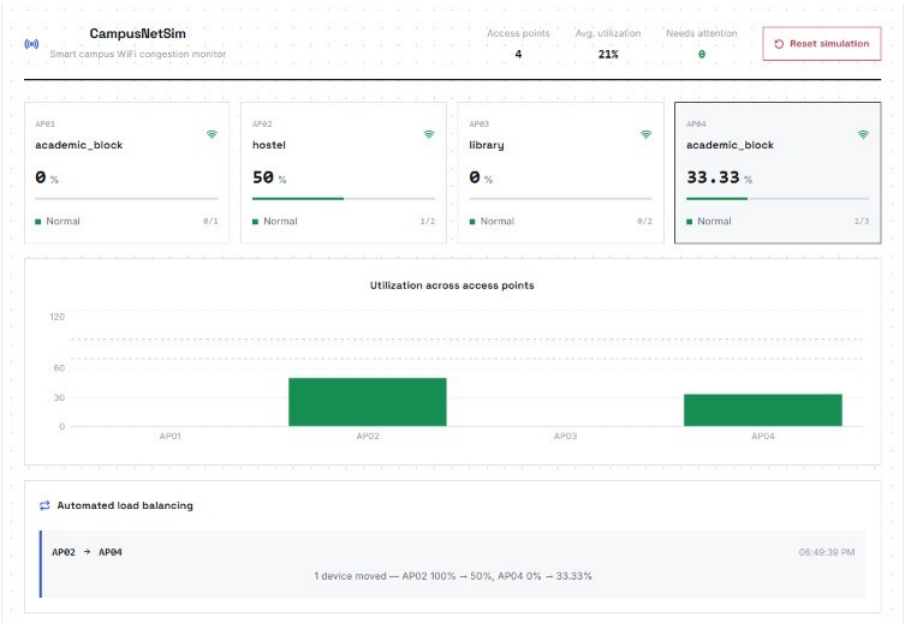
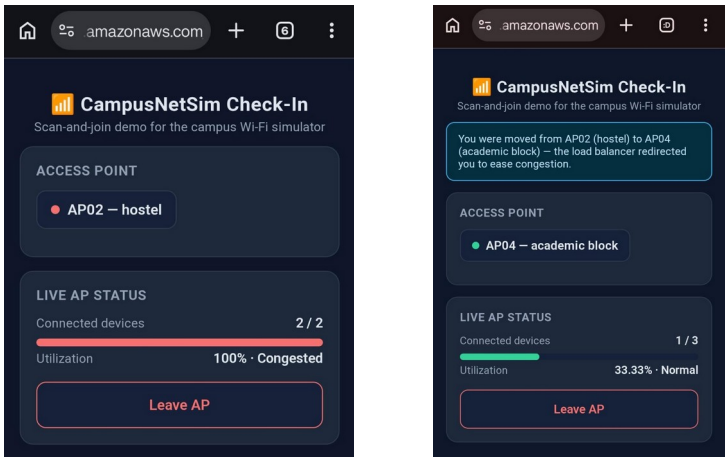


Fig. 6. Dashboard during Run A: AP02 at 50% (1 of 2) and AP04 at 33.33% (1 of 3) after the automated move, with the migration in the load-balancing panel.



Fig. 7. Dashboard during Run A: migration log, connected-device roster with both phones and the AP04 traffic trend rising after the move.



(a) AP02 full before the move (b) after the move to AP04

Fig. 8. Phone check-in page in Run A: (a) AP02 at 2 of 2 devices and 100% (Congested); (b) moments later, the redirect banner and AP04 at 1 of 3 devices (33.33%, Normal).

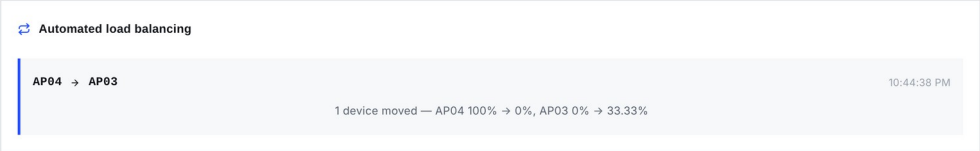


Fig. 9. Migration recorded in Run B: one device moved from AP04 (100% to 0%) to AP03 (0% to 33.33%).

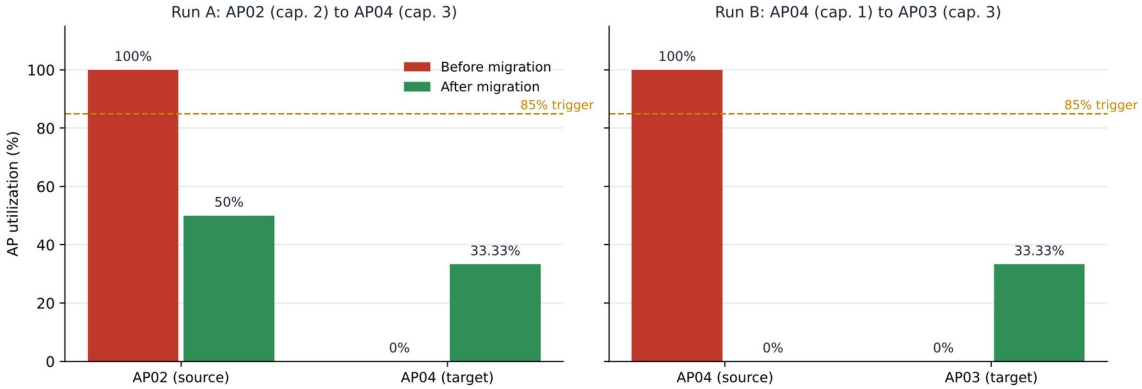


Fig. 10. AP utilization before and after each live migration. In both runs the source fell below the 85% trigger and the target stayed well below it.

TABLE IX
RECORDED MIGRATIONS

Run	Source (C)	Target (C)	k	Source U	Target U
A	AP02 (2)	AP04 (3)	1	100 → 50 %	0 → 33.33 %
B	AP04 (1)	AP03 (3)	1	100 → 0 %	0 → 33.33 %

F. Discussion

Both runs show the intended behaviour: the stream delivered the state change, the balancer picked the AP with the most free capacity, and the source ended below the 85% trigger without pushing the target above it. Because moves are written through POST /events, they obey the same capacity limit and conditional write as phone events, and every move left an auditable Migrations record.

Several limitations remain. First, with the very small capacities used in the demonstrations, a single device changes utilization in large steps (for an AP of capacity 1, from 0% straight to 100%), so the system mostly exercises the full-AP case. Second, the advisory recommendation in EventProcessor prefers an AP in the same building, whereas LoadBalancer chooses the AP with the most free slots campus-wide; the two can name different targets, and the actual move follows the balancer. Third, the balancer calls the public API over HTTPS rather than invoking functions internally, which adds latency and depends on a hard-coded base URL. Fourth, EventProcessor scans the APState table on every event; this is negligible for four APs but grows linearly with the campus. Fifth, no route is authenticated, so PUT /aps/{ap_id}/config and POST /admin/reset are open to anyone who knows the API address, and the check-in page still keeps a static list of valid AP identifiers. Finally, all results come from four simulated APs and a handful of phones; behaviour under many simultaneous check-ins has not been measured.

IX. Conclusion

CampusNetSim demonstrates that AP congestion detection and automated rebalancing can be built entirely from managed serverless services. A DynamoDB Stream on the AP state table drives both load balancing and history logging, so the system reacts to every change without polling and stays idle otherwise. Real phones take part through a QR-code check-in page without installing anything, and the dashboard makes every state change, move and roster update visible within one polling interval. The final build adds run-time capacity reconfiguration with server-side validation and makes event processing concurrency-safe, so DynamoDB is the single source of truth for the campus layout. In two live runs, each initiated by real phone check-ins, the balancer moved a device away from an AP at 100% utilization to the AP with the most free capacity, and all functional tests and latency measurements met expectations.

X. Future Work

The following extensions are planned. (1) Add authentication and authorization, for example JWT authorizers on the HTTP API, starting with the configuration and reset routes. (2) Attach an Amazon EventBridge schedule to TrafficGenerator and run repeatable experiments with normal, peak-hour and congestion scenarios, reporting utilization before and after balancing and the number of migrations. (3) Unify the recommendation and balancing policies, for example by preferring same-building targets in LoadBalancer as well. (4) Replace the per-event table scan with direct key lookups and move the balancer's HTTP calls to direct function invocations or shared code. (5) Use APHistory to forecast when an AP will reach the trigger and rebalance proactively. (6) Detect AP failures and reassign the affected devices. (7) Run load and concurrency tests with many simultaneous check-ins and add CloudWatch alarms and an AWS budget alert.

References

[1] Y. Bejerano, S.-J. Han, and L. Li, "Fairness and load balancing in wireless LANs using association control," *IEEE/ACM Trans. Netw.*, vol. 15, no. 3, pp. 560-573, Jun. 2007.

[2] Y. Bejerano and S.-J. Han, "Cell breathing techniques for load balancing in wireless LANs," *IEEE Trans. Mobile Comput.*, vol. 8, no. 6, pp. 735-749, Jun. 2009.

[3] A. Balachandran, P. Bahl, and G. M. Voelker, "Hot-spot congestion relief in public-area wireless networks," in *Proc. 4th IEEE Workshop Mobile Comput. Syst. Appl. (WMCSA)*, Callicoon, NY, USA, 2002, pp. 70-80.

[4] *IEEE Standard for Information Technology - Telecommunications and Information Exchange Between Systems - Local and Metropolitan Area Networks - Specific Requirements - Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications*, IEEE Std 802.11-2020, 2021.

[5] *IEEE Standard for Information Technology - Part 11, Amendment 8: IEEE 802.11 Wireless Network Management*, IEEE Std 802.11v-2011, 2011.

[6] *IEEE Standard for Information Technology - Part 11, Amendment 2: Fast Basic Service Set (BSS) Transition*, IEEE Std 802.11r-2008, 2008.

[7] E. Jonas *et al.*, "Cloud programming simplified: A Berkeley view on serverless computing," EECS Dept., Univ. California, Berkeley, CA, USA, Tech. Rep. UCB/EECS-2019-3, Feb. 2019.

[8] P. Castro, V. Ishakian, V. Muthusamy, and A. Slominski, "The rise of serverless computing," *Commun. ACM*, vol. 62, no. 12, pp. 44-54, Dec. 2019.

[9] Z. Li, L. Guo, J. Cheng, Q. Chen, B. He, and M. Guo, "The serverless computing survey: A technical primer for design architecture," *ACM Comput. Surv.*, vol. 54, no. 10s, Art. no. 220, 2022.

[10] G. DeCandia *et al.*, "Dynamo: Amazon's highly available key-value store," in *Proc. 21st ACM Symp. Operating Syst. Principles (SOSP)*, Stevenson, WA, USA, 2007, pp. 205-220.

[11] M. A. Ahmed, S. A. Chavez, A. M. Eltamaly, H. O. Garces, A. J. Rojas, and Y.-C. Kim, "Toward an intelligent campus: IoT platform for remote monitoring and control of smart buildings," *Sensors*, vol. 22, no. 23, Art. no. 9045, 2022.

[12] Amazon Web Services, "Serverless architectures with AWS Lambda: Overview and best practices," AWS Whitepaper, 2017. [Online]. Available: https://d1.awsstatic.com/whitepapers/serverless-architectures-with-aws-lambda.pdf

[13] Amazon Web Services, "Using AWS Lambda with Amazon DynamoDB," *AWS Lambda Developer Guide*. [Online]. Available: https://docs.aws.amazon.com/lambda/latest/dg/with-ddb.html (accessed Oct. 23, 2026).

[14] Amazon Web Services, "Change data capture for DynamoDB Streams," *Amazon DynamoDB Developer Guide*. [Online]. Available: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Streams.html (accessed Oct. 23, 2026).

[15] Amazon Web Services, "Using time to live (TTL) in DynamoDB," *Amazon DynamoDB Developer Guide*. [Online]. Available: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/TTL.html (accessed Oct. 23, 2026).

[16] Amazon Web Services, "Optimistic locking with version number," *Amazon DynamoDB Developer Guide*. [Online]. Available: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/BestPractices_OptimisticLocking.html (accessed Oct. 23, 2026).

[17] Amazon Web Services, "Working with HTTP APIs," *Amazon API Gateway Developer Guide*. [Online]. Available: https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api.html (accessed Oct. 23, 2026).

[18] Amazon Web Services, "Deploying an application to Amplify without a Git repository," *AWS Amplify Hosting User Guide*. [Online]. Available: https://docs.aws.amazon.com/amplify/latest/userguide/manual-deploys.html (accessed Oct. 23, 2026).

[19] S. Phadnis and P. J, "CampusNetSim: Cloud-native campus Wi-Fi congestion simulation and load balancing," GitHub repository, 2026. [Online]. Available: https://github.com/pj-06/campusnetsim